10,000벌 의상 Style ID 산출 비용 벤치마크 및 시간 분석

작성일자	대상 시스템	산정 기준	기준 환율
2026-10-13	STMX VTON `/style-ex`	의류 상품 10,000벌 추출	$1 = 1,400원

Anthropic API 정가	현재 배포 (관찰 캐싱)	[방안 A] Anthropic 최적화	[방안 B] Gemini 1.5 Flash
약 812만 원	**약 388만 원**	**약 35 ~ 49만 원**	**약 4 ~ 7만 원**
$5,800 (건당 $0.58)	$2,770 (건당 $0.28, -52%)	Sonnet+Haiku+Batch (-95%)	초가성비 VLM (-99.4% 절감)

01 `/style-ex` 기본 파이프라인 및 토큰 구조 (실측치)

`vton` 의 `/style-ex` 는 상품 사진 1장으로부터 4글자 스타일 아이디(예: `ICSM` , `ETSD`)를 추출하기 위해 3단계 과정을 거칩니다:

INPUT 상품 이미지 (Public URL / Supabase Storage)
단일 의류 정면/대표 상품 이미지 주입

▼

STEP ① 관찰 (Vision API) — `claude-opus-4-8`
- **입력**: 관찰 프롬프트(약 68k) + 이미지
- **출력**: 37개 파라미터 · DM 디테일 · 색상 JSON (약 3.3k)

▼

STEP ② 스코어링 (Local Scoring Engine) — `로컬 JS 엔진 (비용 $0)`
- **서버 내부 연산**: EI · SR · DM 축 스코어링 산출
- 외부 API 호출 없음

▼

STEP ③ 판정 (Judge API) — `claude-opus-5-5 (effort: high)`
- **입력**: 판정 프롬프트(약 12k) + 관찰값 JSON(약 2k) = 14,063 토큰
- **출력**: 축별 근거 JSON + 사고 토큰 (약 5,000 토큰)

▼

★ 최종 Style ID 완성 (4글자 코드, 예: ICSM / ETSD)

건당 토큰 소요 실측치 (기준: `docs/product-style-id-extraction-price.md`)

단계	사용 모델	입력 토큰 (실측)	출력 토큰 (추정)	정가 단가 (1M당)
① 관찰	claude-opus-4-8	68,068	약 2,600 ~ 4,000 (평균 3,300)	$5.00 / $25.00
② 스코어링	로컬 JS 엔진	-	-	$0 (내부 연산)
③ 판정	claude-opus-5-5 (effort: high)	14,063	답안 ~1,200 + 사고 ~3,800 (합 5,000)	$4.00 / $20.00

* 비고: 관찰 단계는 대용량 프롬프트(68k) 캐싱 여부가 전체 비용의 85% 이상을 좌우하며, 판정 단계는 CoT(사고 토큰) 생성이 주요 비용 요인입니다.

02 비용 비교: Claude Code CLI vs Anthropic API (10,000벌)

2.1. Claude Code CLI 사용 시

`1,120만 ~ 2,100만 원+`

Claude Code는 개발용 대화형 에이전트 CLI로, 대량 배치 추출에 적용 시 구조적 비효율로 비용이 폭증합니다:

- **시스템 프롬프트 오버헤드:** 도구 정의(Bash, Read, Edit 등), 안전 수칙 등 고유 시스템 프롬프트만 매 턴 **15,000 ~ 20,000 토큰** 고정 주입.
- **다중 턴 컨텍스트 누적:** 스크립트 실행, 이미지 조회 등 도구 호출 순환마다 이전 대화 이력이 누적되어 건당 **100,000 ~ 150,000 토큰**으로 급증.
- **캐싱 및 배치 할인 제어 불가:** 프롬프트 캐싱 위치의 세밀한 제어가 어렵고, 50% 할인되는 Batch API 적용 불가.
- **Rate Limit 물리적 차단:** Pro/Team 구독 플랜은 5시간당 호출 횟수 제한으로 10,000건 연속 처리가 물리적으로 불가능.

건당: 약 $0.80 ~ $1.50+ → 10,000벌: 약 $8,000 ~ $15,000+

2.2. Anthropic API 직접 호출 케이스

시나리오별 실측

Case 1. 일반 API 호출 (정가, 캐싱 미적용)
- 관찰: 입력 $0.340 + 출력 $0.083 = $0.423
- 판정: 입력 $0.056 + 출력 $0.100 = $0.156

→ 건당 **$0.58** (약 810원) / 10,000벌: **$5,800 (약 812만 원)**

Case 2. 현재 구현 (관찰 프롬프트 캐싱 적용)
- 68k 관찰 프롬프트 5분 내 0.1배 할인($0.50/1M) 적용
- 관찰 입력이 $0.340 → $0.039로 대폭 감소

→ 건당 **$0.28** (약 390원) / 10,000벌: **$2,770 (약 388만 원)**

Case 3. 관찰 + 판정 프롬프트 모두 캐싱 적용
- 판정 프롬프트(12k) 캐싱 시 판정 입력 $0.056 → $0.013 감소

→ 건당 **$0.23** (약 320원) / 10,000벌: **$2,340 (약 328만 원)**

Case 4. Message Batches API (50% 비동기 할인) + 캐싱
- 24시간 비동기 처리 Batch API: 모든 입출력 단가 50% 추가 할인

→ 건당 **$0.12** (약 165원) / 10,000벌: **$1,170 (약 164만 원)**

2.3. 비용 비교 종합 요약 표 (10,000벌 기준)

처리 방식	건당 비용	10,000벌 총비용 (USD)	10,000벌 총비용 (KRW)	비고
Claude Code CLI (대화형 에이전트)	$0.80 ~ $1.50+	**$8,000 ~ $15,000+**	**약 1,120만 ~ 2,100만 원**	오버헤드 극심
Claude Pro/Team 구독 플랜	월 $20 ~ $30	-	-	호출 한도로 불가
Anthropic API (캐싱 미적용 정가)	약 $0.58	$5,800	**약 812만 원**	Opus 4.8 + 5.5 정가
Anthropic API (현재: 관찰 캐싱)	약 $0.28	$2,770	**약 388만 원**	현재 vton 배포 상태
Anthropic API (관찰 + 판정 캐싱)	약 $0.23	$2,340	**약 328만 원**	판정 캐시 추가 적용 시
Anthropic Message Batches API	**약 $0.12**	**약 $1,170**	**약 164만 원**	비동기 배치 50% 할인

💡 **핵심 결론 (Section 2)**

대량 배치 작업에 Claude Code CLI를 사용하는 것은 엔지니어링 에이전트 오버헤드로 인해 비용이 최대 5배 이상 낭비됩니다. 순수 API 환경에서도 캐싱과 Batch API를 결합하면 기존 812만 원에서 164만 원으로 80% 절감 가능합니다.

03 적은 비용으로 10,000개 의상 Style ID를 산출하는 방법 제안

비용 절감 단계별 로드맵 (비용 폭포 분석)

기존 Opus 정가 베이스라인	약 812만 원 기준점 (100%)
→ Anthropic Batch API + 판정 캐싱 결합	약 164만 원 -80% 절감
→ [방안 A] Anthropic 모델 티어 최적화 (Sonnet + Haiku)	약 35만 ~ 49만 원 -95% 절감
→ [방안 B] Google Gemini 1.5 Flash VLM 전환 (적극 권장)	약 4만 ~ 7만 원 -99.4% 파격 절감

[방안 A] Anthropic 최적화 (Cascading)

약 35만 ~ 49만 원

최상위 최고가 모델(Opus) 의존을 탈피하고 작업별 모델 분리:

- **관찰 모델 변경:** `claude-opus-4-8` → `claude-3-5-sonnet` ($3.00 / $15.00)
  - 패션 비전 일치도 95% 이상 유지하면서 비용 절반 이하 절감.
- **판정 모델 변경:** `claude-opus-5-5` → `claude-3-5-haiku` ($0.80 / $4.00)
  - 이미 정형화된 JSON 텍스트와 룰북만 비교하는 작업이므로 Haiku로도 99% 이상 지시 준수 (사고 토큰 생략 가능).
- **Batch API 결합:** 50% 추가 할인으로 **건당 $0.025 ~ $0.035**.

10,000벌 총비용: 약 $250 ~ $350 (약 35만 ~ 49만 원, 95% 절감)

[방안 B] Gemini 1.5 Flash (적극 권장)

약 4만 ~ 7만 원

Vision 입력 및 프롬프트 캐싱 단가가 가장 저렴한 Gemini 어댑터 구축:

- **압도적 캐시 단가 격차:** Opus 입력 $5.00/1M 대비 **Gemini Flash 캐시 읽기 $0.01875/1M** (약 1/260 수준).
- **건당 비용 산출 내역:**
  - 건당 관찰 입력 (캐시 68k): 약 $0.0013
  - 건당 관찰 출력 (3.3k JSON): 약 $0.0010
  - 판정 단계: 로컬 엔진 + Flash 호출 = 건당 $0.001 이하
- **건당 총비용:** 약 **$0.003 ~ $0.005 (약 4 ~ 7원)**

10,000벌 총비용: 약 $30 ~ $50 (약 4만 ~ 7만 원, 99.4% 절감)

비용 절감 방안 비교 분석 매트릭스

비교 항목	기존 파이프라인 (Opus)	[방안 A] Anthropic 최적화	[방안 B] Gemini 1.5 Flash (권장)
사용 모델 구성	Opus 4.8 + Opus 5.5	Sonnet 3.5 + Haiku 3.5	**Gemini 1.5 Flash (Vision + Text)**
1M 토큰 캐시 단가	$0.50 (읽기)	$0.30 (읽기)	**$0.01875 (약 1/260 수준)**
건당 산출 비용	약 $0.28 (약 390원)	약 $0.025 ~ $0.035 (약 40원)	**약 $0.003 ~ $0.005 (약 4~7원)**
10,000벌 총비용	**약 388만 원**	**약 35만 ~ 49만 원 (-95%)**	**약 4만 ~ 7만 원 (-99.4%)**
구현 소요 공수	즉시 사용 가능	프롬프트 미세 조정 (0.5일)	Gemini API 어댑터 모듈 작성 (1일)

04 Gemini 1.5 Flash 전환 시 소요 시간 벤치마크 (10,000벌)

4.1. 1건당 처리 속도 산출 근거

지표	실측 / 산출값	기술적 세부 내역
초기 응답 속도 (TTFT)	약 0.5 ~ 0.8초	68,000 토큰 관찰 프롬프트에 **Context Caching** 적용 시 첫 토큰 즉각 반환
생성 속도 (TPOT)	초당 150 ~ 250 토큰	고속 경량 VLM 아키텍처로 대용량 JSON 스트리밍 출력
1건당 순수 소요 시간	평균 약 11 ~ 14초	**37개 파라미터 JSON 생성 (약 10~13초) + 로컬 스코어링 및 판정 (약 0.5초)**

4.2. 실행 아키텍처별 10,000벌 총 소요 시간 비교

실행 아키텍처 방식	분당 처리량	10,000벌 총 소요 시간	특징 및 운영 권장
1. 비동기 병렬 워커 풀 (25~30개 동시 요청)	약 120 ~ 125건 / 분	약 1시간 20분 ~ 2시간	**즉각적인 결과 확인**이 필요할 때 최적
2. Gemini Batch API (`batchGenerateContent`)	클라우드 분산	약 1시간 ~ 4시간 (SLA: 최대 24시간)	**정기 대규모 배치 최적** **(50% 추가 할인, 완전 무인 백그라운드)**
3. 단순 순차 단일 루프	약 5건 / 분	약 30 ~ 33시간	비추천 (네트워크 단절 위험)

실행 방식 기술 상세 분석:

- **비동기 병렬 워커 풀 (Node.js / Python 스크립트):** 25~30개의 워커가 동시 요청을 전송. 12초마다 25건 완료 시 초당 약 2건(분당 약 125건). 10,000건 ÷ 125건/분 = **80분**. 지수 백오프(Exponential Backoff) 및 재시도를 고려해도 **2시간 이내** 완료.
- **Google Gemini Batch API (강력 추천):** 10,000건의 이미지 URL과 관찰 프롬프트가 담긴 `.jsonl` 파일을 생성하여 1회 업로드. 로컬 서버 프로세스를 계속 켜둘 필요 없이 Google 클라우드 자원으로 일괄 처리 후 결과 다운로드 (실측 보통 **1~4시간 내외** 완료).

4.3. 대규모 배치 시 필수 체크리스트

1 Context Caching 등록

68k 토큰 관찰 프롬프트를 `cachedContents.create` 로 사전 등록 후 배치 요청 시 해당 ID를 참조해야 속도 3배 향상 및 비용 90% 이상 절감.

2 이미지 페이로드 최적화

Base64 인라인 인코딩 대신 Supabase 스토리지 또는 웹 Public URL을 직접 전달하여 네트워크 대역폭 병목 방지.

3 Bulk DB Insert

로컬 `scoring.ts` 로 Style ID 도출 후 PostgreSQL `product_style` 테이블에 1,000건 단위 청크 벌크 삽입.

★ 엔지니어링 최종 제언 (Recommended Next Step)

10,000벌 의상 스타일 추출 시 **방안 B (Google Gemini 1.5 Flash + Batch API)**를 채택할 것을 강력 권장합니다. 비용은 기존 812만 원에서 **약 4만 ~ 7만 원**으로 99.4% 절감되며, 소요 시간 또한 **1~2시간 내외**로 완료할 수 있어 압도적인 운영 효율을 달성할 수 있습니다.